ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN KHOA HỌC MÁY TÍNH



BÁO CÁO ĐỒ ÁN 1

BLOXORZ SOLVER

Môn học: Cơ sở Trí tuệ nhân tạo (CSC14003)

Nhóm sinh viên thực hiện:
24120010 – Trần Văn Tấn Khôi
24120135 – Nguyễn Đức Duy Tân
24120036 – Lê Văn Thành Đạt
24120035 – Lê Thành Đạt
24120070 – Vương Hữu Khang

Giảng viên phụ trách:
ThS. Bùi Duy Đăng
ThS. Nguyễn Ngọc Đức
ThS. Nguyễn Thanh Tình

Mã nguồn GitHub:
zetomis-was-taken/Mixi-Blox

Video Demo YouTube:
https://youtu.be/SQkEpJWEgQo

Thành phố Hồ Chí Minh, Học kỳ 1 – Năm học 2026–2027

Mục lục

1 Kế hoạch thực hiện & Phân chia công việc (Project Planning) 3
1.1 Bảng phân công trách nhiệm & Đánh giá đóng góp . . . 3
1.2 Lộ trình và Tiến độ thực tế . . . 4

2 Mô hình hóa bài toán tìm kiếm (Problem Formulation) 4
2.1 Không gian trạng thái (State Space) . . . 4
2.2 Không gian hành động (Actions) . . . 4
2.3 Mô hình chuyển trạng thái (Transition Model) . . . 4
2.4 Kiểm tra tính hợp lệ & Trạng thái kết thúc . . . 5
2.5 Quản lý trạng thái lặp (Cycle Detection & Visited Set) . . . 5

3 Cài đặt các thuật toán tìm kiếm (Search Algorithms) 5
3.1 Tìm kiếm theo chiều rộng (Breadth-First Search, BFS) . . . 5
3.1.1 Thứ tự FIFO và chiến lược . . . 5
3.1.2 Quan hệ với mô hình chi phí . . . 6
3.1.3 Vòng lặp chung, bộ đếm và giới hạn . . . 6
3.1.4 Kiểm chứng và kết quả . . . 6
3.2 Tìm kiếm theo chiều sâu (Depth-First Search, DFS) . . . 6
3.2.1 Biến thể đã chọn: DFS lặp với ngăn xếp . . . 6
3.2.2 Trạng thái lặp, tính kết thúc và tính tối ưu . . . 7
3.2.3 Kiểm chứng trên bộ dữ liệu . . . 7
3.2.4 Giới hạn và phạm vi kết luận . . . 7
3.3 Tìm kiếm chi phí đồng nhất (Uniform-Cost Search, UCS) . . . 8
3.3.1 Biểu diễn và giao diện dùng chung . . . 8
3.3.2 Hàm chi phí không đồng nhất . . . 8
3.3.3 Thứ tự chi phí và xử lý đường rẻ hơn . . . 8
3.3.4 Vòng lặp, bộ đếm và giới hạn . . . 9
3.3.5 Kiểm chứng tính đúng và giới hạn . . . 9
3.4 Tìm kiếm A* (A-star Search) . . . 9
3.4.1 Thiết kế hàm ước lượng . . . 10
3.4.2 Không đánh giá quá cao và tính nhất quán . . . 10
3.4.3 Giới hạn do dịch chuyển và hướng khối . . . 10
3.4.4 Kiểm chứng và phạm vi kết luận . . . 10

4 Thiết kế kịch bản thực nghiệm (Experimental Setup) 11
4.1 Phân loại bộ dữ liệu kiểm thử (Test Cases) . . . 11
4.2 Môi trường thực nghiệm & Phương pháp đo lường . . . 11

5 Kết quả thực nghiệm & Phân tích đánh giá (Experimental Results) 12
5.1 Bảng số liệu tổng hợp . . . 12
5.2 Biểu đồ so sánh trực quan . . . 14
5.3 Phân tích & Nhận xét chuyên sâu . . . 15

6 Giao diện chương trình & Video Demo (GUI & Demo) 17
6.1 Mô tả giao diện đồ họa (GUI) . . . 17
6.2 Liên kết Video Demo YouTube . . . 17

7 Kết luận (Conclusion) 17
7.1 Kết quả đạt được 17
7.2 Hướng phát triển trong tương lai 18

8 Tài liệu tham khảo & Phụ lục (References & Appendix) 19
8.1 Tài liệu tham khảo 19
8.2 Phụ lục: Công bố sử dụng Trí tuệ Nhân tạo (GenAI Disclosure) 19
8.2.1 Hỗ trợ của trí tuệ nhân tạo trong phần UCS/A* 19

1. Kế hoạch thực hiện & Phân chia công việc (Project Planning)

1.1. Bảng phân công trách nhiệm & Đánh giá đóng góp

Theo quy định đánh giá của đồ án, điểm số cá nhân của mỗi thành viên được tính dựa trên tỷ lệ phần trăm hoàn thành công việc được phân công (Điểm cá nhân = Điểm nhóm × % Đóng góp).

STT	Thành viên	Nhiệm vụ phụ trách chính	% Đóng góp
1	Trần Văn Tấn Khôi (MSSV: 24120010)	- Xây dựng Game Engine core, board parser, rules, advanced tiles. - Xử lý cơ chế Split switch & teleport hai khối cube $1 \times 1 \times 1$. - Viết mục Problem Formalization trong báo cáo.	22%
2	Nguyễn Đức Duy Tân (MSSV: 24120135)	- Phát triển 3D GUI bằng Ursina (Panda3D), board view, block view. - Animation cuộn lăn 12 case, hiệu ứng vỡ ô, cầu đóng mở. - Xây dựng Solver panel & Replay; Thực hiện Video demo YouTube.	21%
3	Lê Văn Thành Đạt (MSSV: 24120036)	- Xây dựng Solver base, SearchProblem, Frontier, visited set. - Cài đặt thuật toán BFS và DFS/IDS có kiểm soát chu trình. - Viết mục thuật toán BFS/DFS trong báo cáo.	19%
4	Lê Thành Đạt (MSSV: 24120035)	- Cài đặt thuật toán UCS (đề xuất & biện minh cost function). - Cài đặt thuật toán A* (thiết kế Heuristic h_1, h_2 & chứng minh). - Viết mục thuật toán UCS/A* trong báo cáo.	21%
5	Vương Hữu Khang (MSSV: 24120070)	- Thiết lập hạ tầng repo, CI linting (Ruff) + testing (Pytest). - Xây dựng module Benchmark đo đạc (time, RAM tracemalloc, nodes, moves). - Thiết lập 10 Test Stages chuẩn 4 nhóm; Soạn khung báo cáo LaTeX. - Viết scripts tự động kiểm tra & đóng gói nộp bài (`tools/package.py`).	17%

Bảng 1: Bảng phân công nhiệm vụ và tỷ lệ đóng góp của 5 thành viên trong nhóm

1.2. Lộ trình và Tiến độ thực tế

Dự án được triển khai qua 5 cột mốc chính:

1. **Tuần 1**: Phân tích yêu cầu, thiết kế cấu trúc dữ liệu bản đồ (JSON format) và lập trình Core Game Engine (kiểm thử bảng chuyển trạng thái Transition Table).
2. **Tuần 2**: Hoàn thiện xử lý các ô nâng cao (Fragile tiles, Bridges, Switches, Split block) và cài đặt 4 thuật toán tìm kiếm (BFS, DFS, UCS, A*).
3. **Tuần 2.5**: Phát triển giao diện người dùng GUI, tích hợp các Solver và tính năng phát lại (replay/animation) lời giải.
4. **Tuần 3**: Thiết kế bộ dữ liệu kiểm thử phân tầng độ khó, chạy thực nghiệm tự động bằng thư viện `benchmark`, vẽ biểu đồ so sánh.
5. **Tuần 3.5**: Hoàn tất Báo cáo kỹ thuật, quay Video thuyết minh trên YouTube và đóng gói mã nguồn nộp bài.

2. Mô hình hóa bài toán tìm kiếm (Problem Formulation)

2.1. Không gian trạng thái (State Space)

Trong trò chơi Bloxorz, không gian trạng thái S bao gồm:

1. **Định hướng và vị trí khối hộp**: Khối có kích thước $1 \times 1 \times 2$, tại mỗi bước tồn tại ở một trong ba trạng thái định hướng O:
   - **Đứng (Standing - S)**: Khối chiếm 1 ô duy nhất tại tọa độ (r, c).
   - **Nằm ngang (Lying Horizontally - H)**: Khối chiếm 2 ô liền kề (r, c) và $(r, c+1)$.
   - **Nằm dọc (Lying Vertically - V)**: Khối chiếm 2 ô liền kề (r, c) và $(r+1, c)$.

   Trường hợp phân tách khối (Split state): Trạng thái lưu tọa độ của cả hai khối lập phương $1 \times 1 \times 1$: (r_1, c_1) và (r_2, c_2) kèm con trỏ chỉ định khối đang được điều khiển.
2. **Trạng thái cầu (Bridges Configuration)**: Giả sử màn chơi có k cây cầu, trạng thái của các cây cầu được biểu diễn dưới dạng tuple hoặc bitmask nhị phân $B = (b_1, b_2, \ldots, b_k)$ với $b_i \in \{0, 1\}$ (0: Đóng / Void, 1: Mở / Sàn).

Do đó, một trạng thái tổng quát được biểu diễn chính xác là một bộ:

$$s = (O, \text{BlockPosition}, B) \tag{1}$$

2.2. Không gian hành động (Actions)

Tại mỗi bước, tập hành động hợp lệ $A = \{\text{Up}, \text{Down}, \text{Left}, \text{Right}\}$. Khi khối bị tách thành 2 khối nhỏ, hành động mở rộng thêm thao tác hoán đổi quyền điều khiển (`SwitchControl`).

2.3. Mô hình chuyển trạng thái (Transition Model)

Bảng 2 mô tả quy luật biến đổi tất định vị trí và định hướng khi thực hiện hành động lăn từ tọa độ tham chiếu (r, c):

Bảng 2: Quy luật chuyển đổi trạng thái khi lăn khối 1×1×2

Trạng thái hiện tại	Up	Down	Left	Right
Đứng $S(r,c)$	$V(r-2,c)$	$V(r+1,c)$	$H(r,c-2)$	$H(r,c+1)$
Nằm ngang $H(r,c)$	$H(r-1,c)$	$H(r+1,c)$	$S(r,c-1)$	$S(r,c+2)$
Nằm dọc $V(r,c)$	$S(r-1,c)$	$S(r+2,c)$	$V(r,c-1)$	$V(r,c+1)$

2.4. Kiểm tra tính hợp lệ & Trạng thái kết thúc

- **Nước đi không hợp lệ (Illegal Move)**: Một hành động lăn bị coi là không hợp lệ nếu bất kỳ phần nào của khối chiếm ô ngoài bản đồ, ô trống (Void), ô cầu đang ở trạng thái Đóng ($b_i = 0$), hoặc khối rơi vào trạng thái Đứng trên ô màu cam dễ vỡ (Fragile Tile).
- **Kiểm tra đích (Goal Test)**: Trạng thái đích đạt được khi và chỉ khi khối ở trạng thái **Đứng (Standing)** tại tọa độ ô mục tiêu (Goal Hole).

2.5. Quản lý trạng thái lặp (Cycle Detection & Visited Set)

Để đảm bảo thuật toán không bị rơi vào chu trình vô hạn, cấu trúc dữ liệu `visited` được cài đặt bằng bảng băm (`set` trong Python) với khóa băm (hashable key) là tuple bất biến:

$$\text{key} = (O, \text{tuple}(\text{pos}), \text{tuple}(B)) \tag{2}$$

3. Cài đặt các thuật toán tìm kiếm (Search Algorithms)

3.1. Tìm kiếm theo chiều rộng (Breadth-First Search, BFS)

BFS sử dụng chung `State`, `Action`, `Board` và vòng lặp tìm kiếm chung của solver cơ sở. Sự khác biệt của BFS nằm ở chính sách lấy các node: hàng đợi FIFO.

3.1.1. Thứ tự FIFO và chiến lược

Đối với BFS, frontier là hàng đợi FIFO. Hai phương thức quan trọng trong hàng đợi gồm `push` và `pop`. Trong đó, `push` sẽ không duyệt lại các node đã gặp và `pop` sẽ lấy phần tử cũ nhất ra khỏi frontier. Các node được đánh dấu là đã gặp tại thời điểm sinh node, không phải lúc mở rộng frontier. Do đó mà mỗi trạng thái chỉ vào frontier đúng một lần nên vòng lặp tìm kiếm không phụ thuộc vào chu trình.

Kiểm tra đích diễn ra khi lấy nút ra, và các nút được lấy theo độ sâu không giảm. Nút đích đầu tiên được lấy ra có độ sâu nhỏ nhất; vì BFS không đọc chi phí, mỗi cạnh trong cây tìm kiếm đúng bằng một bước, nên độ sâu chính là số bước. Do đó BFS bảo đảm tính đầy đủ trên đồ thị hữu hạn và trả về đường ít bước nhất. `bfs.py` chỉ gọi lại `search(stage, caps, frontier=Frontier())`: luật trò chơi nằm ở `Board.successors`, còn `Frontier()` đã là hàng đợi FIFO.

3.1.2. Quan hệ với mô hình chi phí

BFS dùng FIFO và không đọc `step_cost`, nên thuật toán chỉ tối ưu số bước di chuyển, không tối ưu chi phí. Trên các màn chỉ gồm ô thường, mọi bước đi đều có giá trị 1, nên số bước trùng với chi phí và BFS tối ưu cả hai. Trên màn có ô dễ vỡ, chi phí mỗi bước không còn bằng 1 nên hai mục tiêu tách nhau: BFS có thể trả về đường ít bước hơn nhưng đắt hơn UCS (ví dụ ở mục 3.3).

3.1.3. Vòng lặp chung, bộ đếm và giới hạn

Phần cài đặt của BFS gọi `base.search(stage, caps, frontier=Frontier())`, khai báo frontier tường minh và đối xứng với `ucs.py` và `astar.py`, dù `Frontier()` vốn đã là FIFO mặc định. Theo quy ước của #18, `generated` đếm cả các node bị frontier từ chối, `expanded` tăng trước `successors`; `frontier_peak` là số node sống.

Giới hạn thời gian được kiểm tra trước khi lấy nút; giới hạn số nút nằm sau kiểm tra đích và trước khi tăng `expanded`. Mọi kết quả giải được, đạt giới hạn hay không có nghiệm đều trả `Counts`. Bộ giải chỉ dùng đồng hồ để thực thi giới hạn; thời gian và bộ nhớ thực nghiệm do công cụ đo chung phụ trách, nên mục này không nêu số đo. Gọi V và E là số trạng thái và cạnh đạt được: BFS chạy trong $O(V + E)$ thời gian, dùng $O(V)$ bộ nhớ cho tập trạng thái đã gặp và frontier, và không có hàng đợi ưu tiên nên không phát sinh thừa số log.

3.1.4. Kiểm chứng và kết quả

Ba màn có đáp án tính tay cho kết quả 6, 4 và 4 bước, trùng với `expected.optimal_cost`. Với mỗi màn, đường đi trả về được phát lại trên `Board`; trạng thái cuối là đích và chi phí phát lại khớp chi phí bộ giải báo. Trên mười màn chính thức, số bước của BFS được đối chiếu với một phép duyệt theo chiều rộng viết độc lập ngay trong kiểm thử, không đi qua `base.search`; hai kết quả khớp trên cả mười màn.

Hai kiểm thử nhỏ khoá hành vi của frontier. Màn thứ nhất đặt đích là nút con đầu tiên và một nhánh năm bước là nút con thứ hai: FIFO lấy đích ngay và trả một bước, trong khi LIFO sẽ đi hết nhánh và trả sáu bước. Màn thứ hai chứa chu trình $A \to B \to A$: vì trạng thái được đánh dấu ngay khi sinh, A bị từ chối ở lần gặp thứ hai, vòng lặp dừng và mỗi trạng thái chỉ được mở rộng một lần, với ba nút mở rộng và năm nút sinh.

So với UCS, BFS cho cùng số nút mở rộng, số nút sinh và đỉnh frontier trên chín trong mười màn. Ở các màn đó đường ít bước nhất cũng là đường rẻ nhất, nên hai chiến lược duyệt cùng một tập nút dù BFS không đọc chi phí. Màn L2-02 là ngoại lệ: BFS mở rộng 134 nút, sinh 286 nút, đỉnh frontier 9; UCS là 135, 288 và 10. BFS duyệt ít hơn nhưng trả đường đắt hơn, vì mục tiêu số bước và mục tiêu chi phí đã tách nhau từ khi có ô dễ vỡ (Mục 3.3). Đây là kiểm tra tính đúng, không phải số đo thời gian hay bộ nhớ; chưa thể kết luận BFS nhanh hơn hay tốn ít bộ nhớ hơn UCS trước khi có số đo của công cụ chung.

3.2. Tìm kiếm theo chiều sâu (Depth-First Search, DFS)

3.2.1. Biến thể đã chọn: DFS lặp với ngăn xếp

DFS được cài đặt dưới dạng lặp, dùng ngăn xếp LIFO tường minh thay cho đệ quy. Lớp `StackFrontier` kế thừa `Frontier` của vòng lặp chung và chỉ đổi thao tác lấy nút: nút được đưa vào gần nhất được mở rộng trước. Cách này tránh giới hạn độ sâu đệ quy của Python

trên màn lớn và giữ nguyên chính sách trạng thái lặp, bộ đếm cùng các giới hạn của hạ tầng dùng chung. Bộ giải vẫn theo giao diện `solve(stage, caps)` đã đóng băng.

Nhóm chọn DFS thay vì IDS và ghi lại lựa chọn này. Trên mười màn chính thức, DFS trả tổng 701 bước, còn phép duyệt FIFO trong kiểm thử trả 328 bước, tức 2,1 lần; mức tệ nhất là L4-02 với 168 bước so với 24. Độ dài tăng đều như vậy vẫn đọc được như một đặc điểm của chiến lược duyệt sâu, không phải hiện tượng đường đi dài bất thường, nên việc mở rộng lại của IDS không cần thiết để so sánh ở mục thực nghiệm.

3.2.2. Trạng thái lặp, tính kết thúc và tính tối ưu

Mỗi trạng thái được đưa vào ngăn xếp nhiều nhất một lần. `Frontier.push` từ chối khóa đã gặp, kể cả khi nút tương ứng còn đang chờ hoặc đã được mở rộng, nên tập đã gặp chính là cơ chế chặn chu trình. Trên đồ thị hữu hạn, ngăn xếp cạn dần và tìm kiếm kết thúc bằng nghiệm hoặc kết luận không có nghiệm; không nhánh nào quay lại vô hạn.

DFS đầy đủ nhưng không tối ưu. Nó trả nghiệm đầu tiên chạm đích theo thứ tự ngăn xếp, nên cả số bước lẫn chi phí đều có thể lớn hơn nghiệm tối ưu. Trên L1-02, DFS trả 50 bước với chi phí 50, trong khi UCS ở mục 3.3 trả 22 bước với chi phí 22. Vì vậy DFS chỉ dùng để kiểm chứng yêu cầu về duyệt sâu, không dùng thay UCS hay A* khi cần nghiệm tối ưu.

3.2.3. Kiểm chứng trên bộ dữ liệu

Kiểm thử phát lại mọi đường đi qua `Board` và so chi phí với chi phí tính lại, nên đường không hợp lệ hoặc chi phí sai đều bị bắt. Ba màn nhỏ có đáp án tính tay đều được giải với 6, 4 và 4 bước, khớp đáp án. Vì DFS không bảo đảm tối ưu, kiểm thử chỉ ràng buộc độ dài không nhỏ hơn đáp án tối ưu chứ không khẳng định bằng.

Hai bàn tổng hợp kiểm tra đúng phần đặc trưng của DFS. Bàn thứ nhất có chu trình nhưng không có đích: tìm kiếm trả `unsolvable` sau khi mở rộng đúng ba trạng thái, thay vì quay vòng. Bàn thứ hai có hai đường tới đích: DFS đi đường 5 bước trong khi vòng lặp FIFO đi đường 2 bước, xác nhận thứ tự LIFO chứ không chỉ xác nhận có tìm ra nghiệm.

Trên mười màn chính thức, DFS giải được cả mười và mọi đường đi phát lại hợp lệ. DFS mở rộng ít nút hơn UCS ở 8/10 màn vì dừng ngay khi chạm đích, nhưng đỉnh ngăn xếp lại lớn hơn ở cả 10/10 màn.

3.2.4. Giới hạn và phạm vi kết luận

Các số trên là kết quả của bộ giải, không phải phép đo thời gian hay bộ nhớ; phép đo thuộc công cụ chung và chưa được thực hiện ở đây.

Việc ngăn xếp đạt đỉnh lớn hơn không đủ để kết luận DFS dùng nhiều bộ nhớ hơn: cả DFS lẫn UCS đều giữ tập trạng thái đã gặp với $O(V)$ bộ nhớ, và đỉnh frontier chỉ là một phần của mức dùng thực tế. Lý thuyết DFS chỉ giữ một nhánh không còn đúng khi bắt buộc phải có tập đã gặp. Với V trạng thái và E cạnh đạt được, thời gian là $O(V + E)$ vì mỗi trạng thái được mở rộng nhiều nhất một lần; bộ nhớ là $O(V)$ cho tập đã gặp cộng với ngăn xếp.

3.3. Tìm kiếm chi phí đồng nhất (Uniform-Cost Search, UCS)

3.3.1. Biểu diễn và giao diện dùng chung

Trạng thái `State(cubes, bridges)` lưu các ô khối đang chiếm và cấu hình cầu. Khóa chuẩn hóa chứa cả hai trường; hai lập phương sau khi tách được sắp theo vị trí. Hành động `(cube_index, direction)` chỉ định lập phương trong trạng thái nguồn, nên phát lại đường đi phải đọc chỉ số theo từng trạng thái. Bộ giải dùng `Board.successors` và không cài lại luật trò chơi.

3.3.2. Hàm chi phí không đồng nhất

Gọi $O(s)$ là tập ô bị chiếm trong trạng thái s và F là tập ô dễ vỡ. Chi phí của cạnh hợp lệ (s, a, s') được định nghĩa:

$$c(s, a, s') = 1 + \left|\big(O(s') \setminus O(s)\big) \cap F\right|. \tag{3}$$

Mỗi lần lăn có chi phí nền 1; mỗi ô dễ vỡ mới tiếp xúc cộng thêm 1. Khối chiếm nhiều nhất hai ô nên chi phí thuộc $\{1, 2, 3\}$ và $c_{\min} = 1 > 0$. Chính sách nằm tại `bloxorz.core.costs`; `Board.step_cost` chỉ gọi chính sách này. Điều chỉnh trọng số không đòi hỏi sửa luật chuyển trạng thái hoặc vòng lặp tìm kiếm.

Phụ phí thể hiện ưu tiên hạn chế tiếp xúc bề mặt dễ vỡ, theo ví dụ của đề bài. Đây là mục tiêu tối ưu được thiết kế, không phải xác suất rơi: trò chơi vẫn xác định. Hệ số 1 tạo đánh đổi dễ giải thích, trong đó tránh một tiếp xúc tương đương giảm một lần lăn thường; không lựa chọn hệ số bằng cách tối ưu kết quả thực nghiệm. Lập phương đứng yên trên ô dễ vỡ không làm tăng chi phí mọi bước của lập phương còn lại. Đích dịch chuyển được xét như tiếp xúc mới; cầu và công tắc không có phụ phí riêng.

Trong mười màn chính thức hiện tại, phụ phí chỉ phát sinh trên L2-01 và L2-02. Tám màn còn lại có chi phí đồng nhất bằng 1, nên UCS có cùng thứ tự với FIFO khi dùng cách phá hòa hiện tại. Chính sách này không bảo đảm đường UCS rẻ hơn đường ít bước nhất trên mọi màn; tác động cần được đối chiếu theo từng màn.

Hợp đồng hành động hiện hành không có cạnh đổi điều khiển độc lập. Vì vậy, không thêm cạnh chi phí 0 hoặc 1 cho việc chọn lập phương; mọi cạnh di chuyển thực sự vẫn có chi phí ít nhất $c_{\min}$. Tổng chi phí g là tổng trọng số cạnh, còn độ dài lời giải là `len(path)`; hai đại lượng không được đánh đồng.

Đối với hàm ước lượng dựa trên khoảng cách, cận số bước phải được nhân với $c_{\min}$ để thành cận chi phí. Chẳng hạn, trên màn không dịch chuyển, nếu d giảm nhiều nhất hai ô sau mỗi lần lăn thì $h = c_{\min} d/2$ không vượt chi phí còn lại. Khi có dịch chuyển, lập luận hình học này cần được thay thế bằng một cận đã chứng minh; $h = 0$ luôn là lựa chọn an toàn. Phần A* sẽ trình bày thiết kế và kiểm chứng cụ thể.

3.3.3. Thứ tự chi phí và xử lý đường rẻ hơn

UCS lấy nút có chi phí tích lũy g nhỏ nhất từ hàng đợi ưu tiên (priority queue). Lớp `CostFrontier` quản lý bảng chi phí tốt nhất theo khóa trạng thái, các nút đang chờ và hàng đợi `heapq`. Khi phát hiện đường rẻ hơn, nút được đưa lại vào hàng đợi, kể cả khi trạng thái từng được mở rộng. Đường có chi phí bằng hoặc lớn hơn chi phí tốt nhất bị từ chối.

Mỗi trạng thái đang chờ có một nút hiện hành. Bản ghi cũ trong hàng đợi bị bỏ qua khi lấy ra nếu không còn là nút hiện hành; không tăng số nút mở rộng. Độ dài frontier chỉ đếm

nút hiện hành, không đếm bản ghi cũ. Khi chỉ còn bản ghi cũ, frontier được coi là rỗng; điều này tránh trả kết quả đạt giới hạn giả sau khi toàn bộ trạng thái đã được duyệt.

Khóa ưu tiên gồm g, giá trị phá hòa và thứ tự chèn tăng dần, nên kết quả lặp lại và không so sánh trực tiếp các trạng thái. Liên kết cha giữ hành động nguồn để dựng lại đường đi. Kiểm tra đích diễn ra khi lấy nút hiện hành ra. Vì mọi chi phí dương, nghiệm được trả có tổng chi phí tối ưu. Trên đồ thị hữu hạn và không áp giới hạn, thuật toán kết thúc bằng nghiệm hoặc kết luận không có nghiệm. Thuật toán kiểm tra chi phí hữu hạn và cận tối thiểu khi frontier nhận một cạnh mới.

3.3.4. Vòng lặp, bộ đếm và giới hạn

UCS gọi `base.search(stage, caps, frontier=...)` theo hạ tầng chung của nhóm. Vòng lặp quản lý bộ đếm và giới hạn; frontier sở hữu chính sách trạng thái lặp và thứ tự lấy nút. Không thay đổi mã base hoặc giao diện `solve(stage, caps)` đã đóng băng.

Theo quyết định của người phụ trách mục #18, `generated` đếm nút gốc cộng mọi trạng thái kế tiếp được đưa ra cho frontier, kể cả khi frontier từ chối vì trùng hoặc đắt hơn. `expanded` tăng ngay trước khi gọi `successors`. `frontier_peak` là số nút hiện hành đang chờ lớn nhất. Các quy ước này được áp dụng chung cho cả thuật toán sau này.

Giới hạn thời gian được kiểm tra trước khi lấy nút; giới hạn nút nằm sau kiểm tra đích và trước khi tăng số nút mở rộng. Mọi kết quả giải được, đạt giới hạn và không có nghiệm đều trả `Counts`. Đồng hồ trong bộ giải chỉ thực thi giới hạn; thời gian và bộ nhớ thực nghiệm do công cụ đo chung phụ trách. Màn chơi đã được nạp và biên dịch trước lời gọi bộ giải.

3.3.5. Kiểm chứng tính đúng và giới hạn

Kiểm thử so sánh với ba màn có đáp án tính tay, phát lại đường đi, kiểm tra giới hạn, trạng thái không có nghiệm, đường rẻ hơn, bản ghi cũ và số nút hiện hành. Trên mười màn chính thức, đồ thị được duyệt độc lập và đảo cạnh để tính chi phí tối ưu còn lại; chi phí UCS từ đầu được đối chiếu với kết quả đó. Tổng cộng đồ thị chứa 2 946 trạng thái đạt được.

Trên L2-02, vòng lặp FIFO dùng trong kiểm thử tìm đường 46 bước, chi phí 64; UCS tìm đường 49 bước, chi phí 58. Ví dụ minh họa tối ưu chi phí có thể cần nhiều bước hơn tối ưu số bước. Đây là kiểm tra tính đúng, không phải số đo thời gian hay bộ nhớ.

Gọi V, E là số trạng thái và cạnh đạt được. Cài đặt dùng bản ghi thay thế có thời gian $O((V+E)\log(V+E))$ và bộ nhớ $O(V+E)$. Số nút hiện hành được đếm riêng, còn bản ghi cũ vẫn chiếm bộ nhớ cho tới khi bị bỏ qua hoặc bộ giải kết thúc. Chưa suy ra kết luận hiệu năng khi chưa thực hiện đo đạc.

3.4. Tìm kiếm A* (A-star Search)

A* sử dụng cùng chi phí tại Công thức (3), ưu tiên $f(s) = g(s) + h(s)$, với h là hàm ước lượng (heuristic). Lớp `AStarFrontier` kế thừa frontier chi phí của UCS và chỉ đổi hàm ưu tiên; cơ chế cải thiện đường đi và mở lại trạng thái được dùng chung, bảo đảm một trạng thái từng gặp vẫn có thể nhận đường rẻ hơn. Các bộ giải giữ nguyên giao diện `solve(stage, caps)` và kiểu kết quả đã đóng băng.

3.4.1. Thiết kế hàm ước lượng

Trên màn không có công tắc tách, đặt:

$$d(s) = \min_{(r,c)\in O(s)} \big(|r - r_G| + |c - c_G|\big), \tag{4}$$

$$h(s) = \frac{c_{\min}\, d(s)}{2}. \tag{5}$$

Khoảng cách được lấy từ ô bị chiếm gần đích nhất, không từ một tọa độ tham chiếu có thể làm quá cận. Dữ liệu tĩnh được kiểm tra một lần để tạo hàm ước lượng, không quét lưới trên từng nút.

3.4.2. Không đánh giá quá cao và tính nhất quán

Không đánh giá quá cao (admissibility). Một lần lăn giảm d nhiều nhất hai ô. Bỏ qua hướng khối, vật cản, cầu và ô dễ vỡ chỉ làm bài toán dễ hơn, nên cần ít nhất $d(s)/2$ bước để tới đích. Mỗi bước tốn ít nhất $c_{\min}$, suy ra $h(s) \le h^*(s)$, với h^* là chi phí tối ưu còn lại. Lập luận dùng cận chi phí của chính mô hình không đồng nhất đang chạy.

Nhất quán (consistency). Với mỗi cạnh không dịch chuyển, $d(s) \le d(s') + 2$, do đó:

$$h(s) \le h(s') + c_{\min} \le h(s') + c(s, a, s'). \tag{6}$$

Tại đích, $h = 0$. Mọi phụ phí đều không âm nên bất đẳng thức vẫn đúng trên các bước có tiếp xúc ô dễ vỡ.

3.4.3. Giới hạn do dịch chuyển và hướng khối

Công tắc tách có thể dịch chuyển xa tùy ý, phá vỡ giới hạn hai ô. Vì vậy dùng $h = 0$ trên *mọi trạng thái của màn có công tắc tách*, kể cả trước khi tách. Phương án này không đánh giá quá cao và nhất quán, nhưng làm A* trở thành UCS. Chỉ tắt hàm ước lượng sau khi khối đã tách là không đủ vì chính cạnh kích hoạt dịch chuyển cũng có thể làm giảm khoảng cách rất lớn.

Không cộng tùy ý một đơn vị phạt khi khối nằm. Nếu khối nằm ngay cạnh đích và có thể đứng lên đích trong một bước chi phí 1, khoảng cách ô gần nhất là 1, nên $h = 1/2$. Cộng thêm 1 cho hướng nằm tạo ước lượng $3/2 > 1$, vượt chi phí thật. Vì vậy phần cài đặt không sử dụng hạng phạt hướng chưa được chứng minh.

3.4.4. Kiểm chứng và phạm vi kết luận

Kiểm thử phát lại đường đi, so sánh chi phí UCS/A*, kiểm tra giới hạn, trường hợp không có nghiệm, bản ghi cũ và mở lại nút đã mở rộng dưới một hàm ước lượng không đánh giá quá cao nhưng không nhất quán. Trên ba màn nhỏ có đáp án tính tay, chi phí còn lại được tính độc lập ở mọi trạng thái có đường tới đích và đối chiếu với h. Kiểm tra mở rộng trên mười màn chính thức duyệt 2 946 trạng thái đạt được, tạo đồ thị cạnh ngược để tính chi phí tối ưu còn lại và kiểm tra tính nhất quán trên mọi cạnh.

Ví dụ kiểm tra tính đúng trên L2-02: phép duyệt ít bước nhất chỉ nằm trong kiểm thử tìm đường 46 bước, chi phí 64; UCS/A* chọn đường 49 bước, chi phí 58. Đây là bằng chứng cho sự khác nhau giữa mục tiêu số bước và mục tiêu chi phí, không phải số liệu đo thời gian hoặc bộ nhớ. Các chứng minh ở trên áp dụng cho mô hình đã nêu; phép duyệt hữu hạn là kiểm chứng bổ sung, không thay thế chứng minh cho mọi màn có thể được tạo.

Với hàm ước lượng nhất quán đang dùng, mỗi trạng thái được mở rộng nhiều nhất một lần hợp lệ. Gọi V, E là số trạng thái/cạnh đạt được, thời gian dùng hàng đợi ưu tiên là $O((V+E)\log(V+E))$, bộ nhớ $O(V+E)$ kể cả bản ghi thay thế. Không suy ra A* luôn nhanh hơn UCS: độ chặt của cận và chi phí tính hàm ước lượng đều ảnh hưởng kết quả. Trên màn tách, hai thuật toán có cùng chiến lược vì $h = 0$. Đánh giá hiệu năng toàn nhóm chờ bộ công cụ đo đạc và phiên bản tích hợp cuối; phần này không đưa số đo chưa được thực hiện.

4. Thiết kế kịch bản thực nghiệm (Experimental Setup)

4.1. Phân loại bộ dữ liệu kiểm thử (Test Cases)

Bộ dữ liệu gồm 10 màn chơi chuẩn được chia thành 4 cấp độ phức tạp tăng dần (tỷ lệ 3/2/3/2 theo thiết kế kiểm thử đồ án):

- **Nhóm 1 (Cơ bản - L1, 3 màn)**: `L1-01_long_corridor`, `L1-02_two_flights`, `L1-03_side_rooms`. Bản đồ chỉ gồm các ô sàn kiên cố tiêu chuẩn, kiểm tra khả năng điều hướng cơ bản và không gian trạng thái định hướng (Standing, Horizontal, Vertical).

- **Nhóm 2 (Ô dễ vỡ - L2, 2 màn)**: `L2-01_fragile_shelves`, `L2-02_fragile_ring`. Bản đồ chứa các dải ô cam dễ vỡ (Fragile tiles), buộc khối chỉ được phép lăn qua ở tư thế nằm (ngang/dọc), khối sẽ rơi và thua ngay nếu đứng thẳng lên ô này.

- **Nhóm 3 (Cầu & Công tắc - L3, 3 màn)**: `L3-01_bridge_under_a_heavy_switch`, `L3-02_a_one-way_bridge`, `L3-03_two_heavy_switches`. Bản đồ tích hợp các công tắc mềm (Soft switch - kích hoạt bởi mọi tư thế) và công tắc nặng (Heavy switch - chỉ kích hoạt khi khối đứng thẳng) để bật/tắt các nhịp cầu (Bridges).

- **Nhóm 4 (Phức tạp & Tách khối - L4, 2 màn)**: `L4-01_split_into_the_far_room`, `L4-02_split_across_the_middle`. Bản đồ diện tích lớn chứa công tắc phân tách (Split switch), tách khối $1 \times 1 \times 2$ thành hai khối lập phương $1 \times 1 \times 1$ điều khiển độc lập qua các khu vực riêng biệt trước khi tái hợp nhất (Merge).

4.2. Môi trường thực nghiệm & Phương pháp đo lường

- **Cấu hình phần cứng**: Vi xử lý Intel x86-64 (Family 6, Model 58 – thế hệ Ivy Bridge), RAM 16GB.

- **Hệ điều hành & Nền tảng**: Windows 10 64-bit (build 10.0.19045), Python 3.11.2 (CPython).

- **Phương pháp đo đạc (Methodology)**:

  - **Thời gian thực thi**: Sử dụng bộ đếm xung độ chính xác cao `time.perf_counter`. Để triệt tiêu nhiễu từ hệ thống và cơ chế thu hồi bộ nhớ (garbage collection), mỗi thuật toán được đo độc lập 3 lần trên từng màn chơi và lấy giá trị trung vị (*median-of-3*).
  - **Bộ nhớ tiêu thụ**: Sử dụng module `tracemalloc.get_traced_memory` đo lượng RAM đỉnh (*peak memory consumption*) trong suốt quá trình tìm kiếm.

– **Khởi tạo lại bộ đo**: Mỗi lượt đo đều gọi `gc.collect()` trước khi bật `tracemalloc` và bấm giờ, nên bộ nhớ còn sót từ lượt trước không lọt vào số đo của lượt sau. Ba lượt chạy liên tiếp được thực hiện trong cùng một tiến trình, và giá trị báo cáo là trung vị của ba lượt.

5. Kết quả thực nghiệm & Phân tích đánh giá (Experimental Results)

5.1. Bảng số liệu tổng hợp

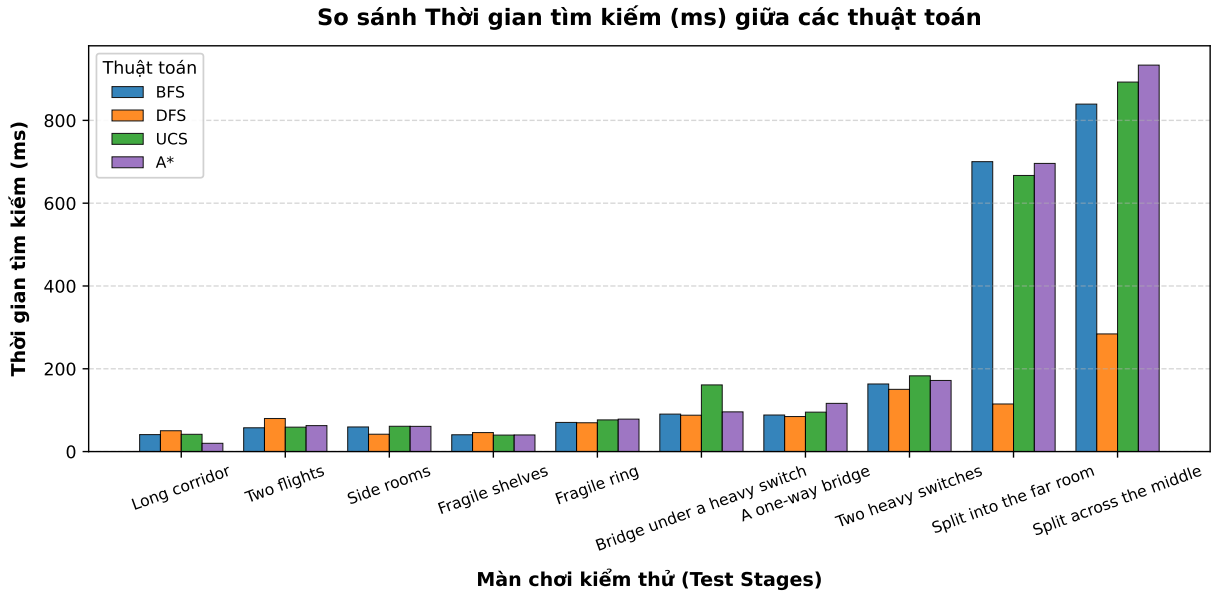
Bảng 3 tổng hợp kết quả đo đạc thực tế của 4 thuật toán tìm kiếm (BFS, DFS, UCS, A*) trên toàn bộ 10 màn chơi kiểm thử:

Bảng 3: Kết quả thực nghiệm bốn thuật toán trên mười màn chơi. Tiền tố L1–L4 lần lượt ứng với bốn nhóm: cơ bản, ô dễ vỡ, cầu & công tắc, tách khối; bảng ký hiệu phía trên ánh xạ từng mã sang tên màn chơi.

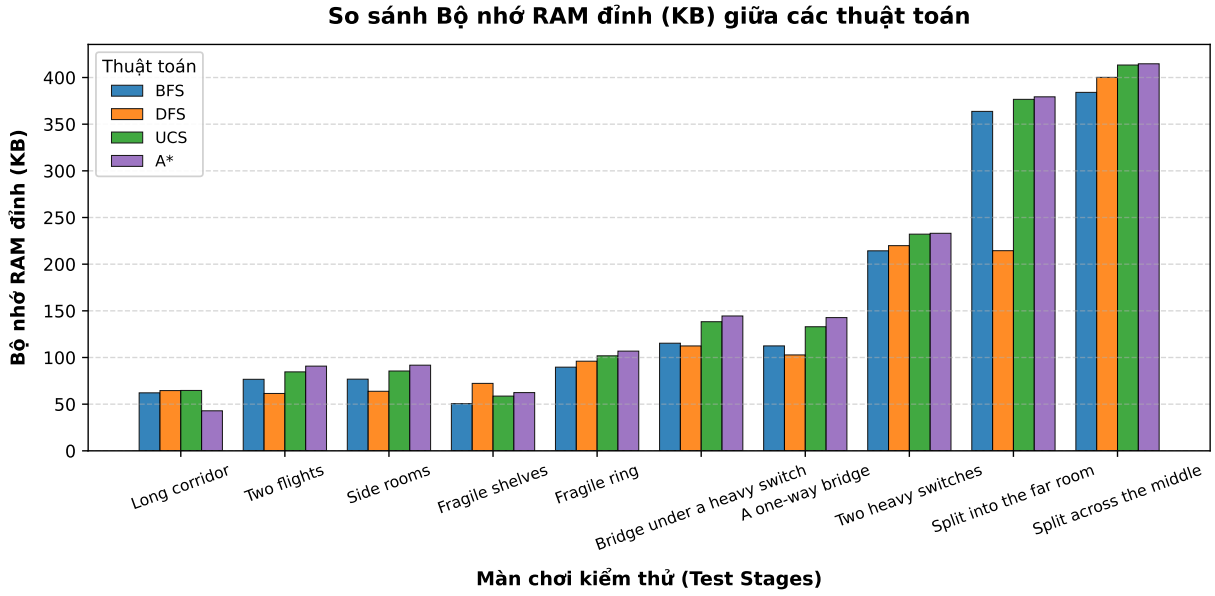
Mã	**Màn chơi**	**Mã**	**Màn chơi**
L1-01	Long corridor	L3-01	Bridge under a heavy switch
L1-02	Two flights	L3-02	A one-way bridge
L1-03	Side rooms	L3-03	Two heavy switches
L2-01	Fragile shelves	L4-01	Split into the far room
L2-02	Fragile ring	L4-02	Split across the middle

Mã	**Thuật toán**	**Thời gian (ms)**	**RAM (KB)**	**Số nút duyệt**	**Số bước đi**
L1-01	BFS	41.03	62.13	64	7
	DFS	50.36	64.57	82	7
	UCS	41.71	64.69	64	7
	A*	19.98	42.89	27	7
L1-02	BFS	57.47	76.63	103	22
	DFS	79.89	61.46	58	50
	UCS	58.98	84.59	103	22
	A*	62.72	90.75	102	22
L1-03	BFS	59.44	76.79	106	35
	DFS	41.90	63.80	77	51
	UCS	61.11	85.53	106	35
	A*	60.88	91.77	105	35
L2-01	BFS	40.61	50.59	69	28
	DFS	45.92	72.26	83	39
	UCS	39.83	58.64	69	28
	A*	40.14	62.41	69	28
L2-02	BFS	70.39	89.64	134	46
	DFS	69.62	96.02	129	57
	UCS	76.51	101.80	135	49
	A*	78.35	106.83	130	49
L3-01	BFS	90.50	115.30	177	44
	DFS	87.86	112.38	153	69
	UCS	161.01	138.35	177	44
	A*	95.92	144.49	177	44
L3-02	BFS	88.30	112.42	186	47
	DFS	84.67	102.75	168	49
	UCS	95.23	132.96	186	47
	A*	116.42	142.77	186	47
L3-03	BFS	163.27	214.30	320	65
	DFS	150.32	219.86	306	113
	UCS	182.95	232.16	320	65
	A*	171.78	233.07	298	65
L4-01	BFS	700.31	363.70	532	10
	DFS	114.96	214.46	114	98
	UCS	666.94	376.60	532	10
	A*	696.04	379.31	532	10
L4-02	BFS	839.30	384.07	741	24
	DFS	284.19	400.15	275	168
	UCS	892.54	413.38	741	24
	A*	933.49	414.70	741	24

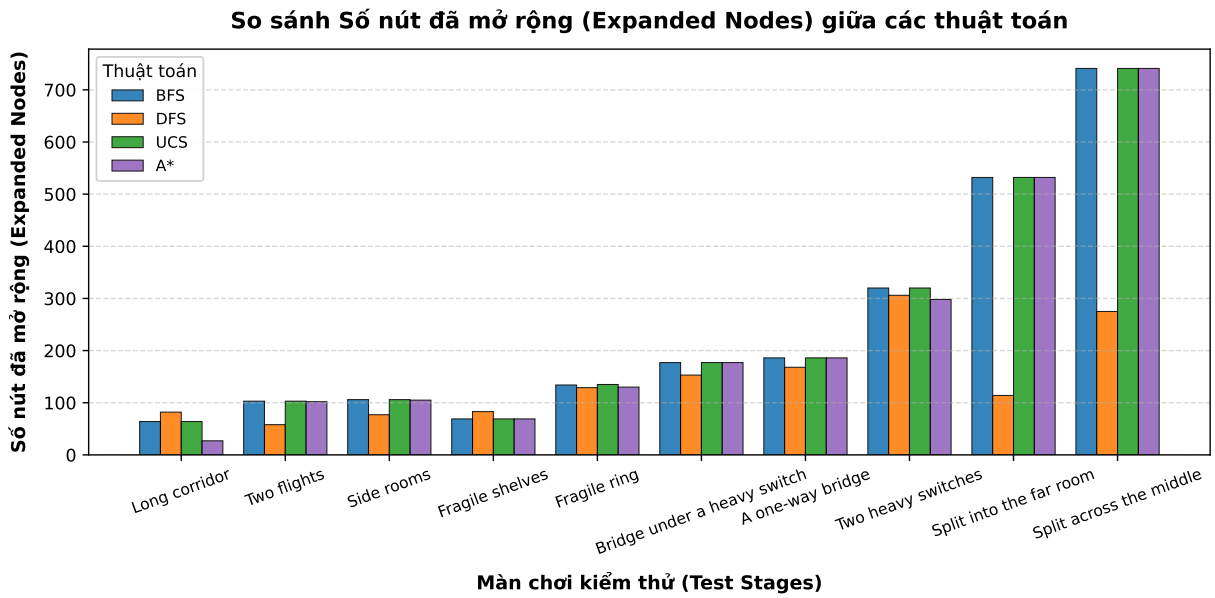
5.2. Biểu đồ so sánh trực quan



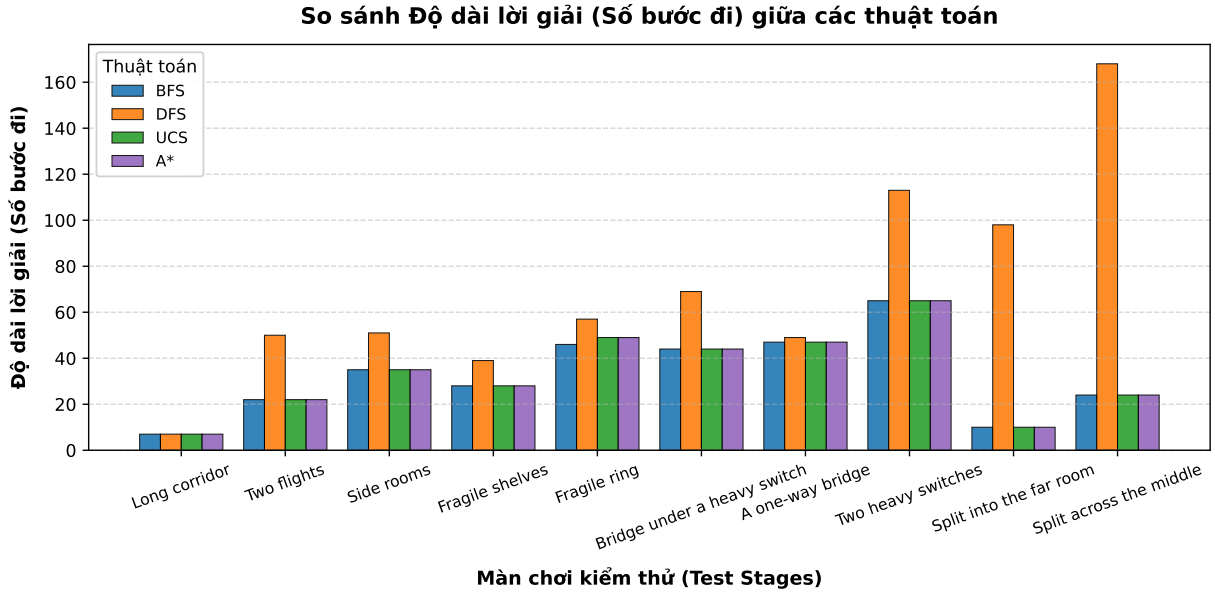
Hình 1: Thời gian thực thi (ms) giữa các thuật toán qua các màn chơi



Hình 2: Bộ nhớ RAM đỉnh (KB) giữa các thuật toán qua các màn chơi



Hình 3: Số nút mở rộng (Nodes) giữa các thuật toán qua các màn chơi



Hình 4: Số bước đi của lời giải (Moves) giữa các thuật toán qua các màn chơi

5.3. Phân tích & Nhận xét chuyên sâu

Từ dữ liệu thực nghiệm tại Bảng 3 và các Hình 1–4, nhóm rút ra các phân tích chuyên sâu sau:

1. **Về số lượng nút mở rộng (Expanded Nodes) và hàm Heuristic**:
   - **A* với hàm heuristic**: Hàm heuristic hướng đích thể hiện sức mạnh rõ nét trên các màn chơi có cấu trúc mở. Ví dụ tiêu biểu ở màn `Long corridor` (L1), A* chỉ cần mở rộng **27 nút** so với **64 nút** của BFS và UCS (tiết kiệm đến **57.8%** không gian tìm kiếm).

- **Ảnh hưởng của chướng ngại vật & Cầu**: Ở các màn chơi thuộc nhóm L3 (cầu và công tắc) và L4 (tách khối), không gian trạng thái bị phân mảnh bởi điều kiện chuyển đổi (phải đi ngược hướng đích để đạp công tắc mở cầu). Khi đó hàm khoảng cách Manhattan đơn thuần không thể định hướng xuyên qua khoảng trống, khiến số nút mở rộng của A* và UCS tiệm cận nhau (ví dụ tại `Bridge under a heavy switch`: BFS, UCS và A* đều mở rộng 177 nút; tại `Split across the middle`: cả ba cùng duyệt 741 nút).
- **BFS và UCS**: Do bản chất duyệt theo đường mức đẳng chi phí (*contour-based search*), BFS và UCS luôn mở rộng một rẻ quạt trạng thái rộng về mọi hướng.
- **Đặc tính của DFS**: DFS có hành vi tìm kiếm bất định do phụ thuộc hoàn toàn vào thứ tự nhánh con trong tập kế thừa (Left, Right, Up, Down). Khi may mắn đâm thẳng vào nhánh dẫn tới đích sớm, DFS mở rộng rất ít nút (ví dụ chỉ 114 nút ở màn `Split into the far room` so với 532 nút của BFS/UCS/A*). Ngược lại, nếu chọn sai nhánh, DFS duyệt sâu vào các ngõ cụt gây lãng phí bộ nhớ và thời gian.

2. **Về tính tối ưu của lời giải (Optimality) và hàm chi phí**:

- **BFS**: Luôn bảo đảm tìm được lời giải có **số bước đi (Moves) ít nhất** trên mọi màn chơi (ví dụ 7 bước ở L1-01, 22 bước ở L1-02, 10 bước ở L4-01).
- **UCS và A***: Luôn bảo đảm tối ưu hóa theo **tổng chi phí hành động** $g(n)$. Điểm khác biệt mấu chốt được kiểm chứng rõ rệt ở màn `Fragile ring` (L2):
  - BFS tìm đường ngắn nhất về số bước là **46 bước**, nhưng chấp nhận lăn qua các ô mỏng fragile.
  - UCS và A* tìm đường có **49 bước** (dài hơn 3 bước) nhằm né tránh các ô fragile có trọng số chi phí cao (cost = 3), chứng minh hàm chi phí và hàng đợi ưu tiên của UCS/A* hoạt động hoàn toàn chính xác theo đúng mô hình bài toán.
- **DFS**: Tuyệt đối **không đảm bảo tính tối ưu**. Lời giải của DFS thường xuyên vòng vèo và dư thừa bước đi nghiêm trọng: ở màn `Two flights` mất 50 bước (so với tối ưu 22 bước), ở màn `Split into the far room` mất 98 bước (gấp gần 10 lần mức tối ưu 10 bước), và ở màn `Split across the middle` mất tới 168 bước (so với tối ưu 24 bước).

3. **Về thời gian thực thi (Runtime) và bộ nhớ RAM đỉnh (Peak Memory)**:

- **Thời gian thực thi**: Cả 4 thuật toán đều hoàn thành việc giải trong thời gian dưới 1 giây trên toàn bộ 10 màn chơi. Thời gian tăng dần theo độ phức tạp của bản đồ: L1 từ 19.98 đến 79.89 ms, L2 từ 39.83 đến 78.35 ms, L3 từ 84.67 đến 182.95 ms. Ở L4, nơi trạng thái tách thành hai khối $1 \times 1 \times 1$ làm không gian trạng thái lớn hơn hẳn, BFS, UCS và A* cần từ 666.94 đến 933.49 ms, còn DFS chỉ cần từ 114.96 đến 284.19 ms vì không duyệt hết không gian trạng thái.
- **Bộ nhớ RAM đỉnh**: Đo lường bằng `tracemalloc` cho thấy mức tiêu thụ RAM tỉ lệ thuận với số trạng thái sinh ra trong tập `visited` và hàng đợi `Frontier`. Mức RAM đỉnh duy trì ổn định trong khoảng **43 KB đến 415 KB**.
- Dù DFS trên lý thuyết chỉ cần lưu nhánh hiện tại trên Stack, nhưng do phải duy trì tập `visited` đầy đủ để triệt tiêu vòng lặp vô hạn (Cycle Prevention), mức

RAM thực tế của DFS vẫn xấp xỉ BFS và UCS (ví dụ ở màn L4-02 là 400.15 KB so với 384.07 KB của BFS).

6. Giao diện chương trình & Video Demo (GUI & Demo)

6.1. Mô tả giao diện đồ họa (GUI)

Chương trình được phát triển giao diện 3D tương tác hoàn chỉnh bằng thư viện Ursina (dựa trên nền tảng Panda3D), bao gồm:

- Bàn cờ hiển thị dạng 3D trực quan, phân biệt rõ màu sắc và vật liệu của từng loại ô (Sàn tiêu chuẩn, Ô cam dễ vỡ, Cầu đóng/mở linh hoạt, Công tắc hình tròn Soft Switch và hình chữ X Heavy Switch).
- Khối hộp hiển thị chuyển động lăn 3D mượt mà (Smooth roll animation qua 12 trường hợp xoay góc).
- Bảng điều khiển tích hợp: Chọn màn chơi trong bộ Test Set, New Game / Restart, và 4 nút kích hoạt các thuật toán tìm kiếm (BFS, DFS, UCS, A*) kèm bảng số liệu thống kê thời gian, bộ nhớ, số bước.
- Chế độ phát lại (Replay mode): Cho phép Play, Pause, Step từng bước hoặc điều chỉnh tốc độ lăn của lời giải.

6.2. Liên kết Video Demo YouTube

Video trình diễn đầy đủ các tính năng của chương trình và giải các màn chơi mẫu:

Link Video YouTube: `https://youtu.be/SQkEpJWEgQo`

7. Kết luận (Conclusion)

7.1. Kết quả đạt được

Qua quá trình nghiên cứu, mô hình hóa và triển khai đồ án Bloxorz Solver, nhóm đã hoàn thành toàn diện các mục tiêu đề ra:

1. **Mô hình hóa bài toán chính xác**: Xây dựng Game Engine mô phỏng trọn vẹn quy luật không gian trạng thái của trò chơi Bloxorz, bao gồm đầy đủ các cơ chế nâng cao như ô dễ vỡ (Fragile tiles), hệ thống cầu đóng/mở (Bridges) điều khiển bởi công tắc mềm (Soft switches) và công tắc nặng (Heavy switches), cùng cơ chế phân tách và tái hợp nhất khối (Split & Merge).
2. **Cài đặt và kiểm chứng 4 thuật toán tìm kiếm**:
   - Cài đặt thành công BFS, DFS (kèm tập `visited` triệt tiêu chu trình lặp), UCS và A*.

- Đề xuất và biện minh mô hình hàm chi phí hành động thực tế nhằm phản ánh rủi ro khi tương tác với các ô đặc biệt (ô cam dễ vỡ gán chi phí 3, roll thường gán 1).
- Thiết kế và chứng minh tính admissible, consistent của hàm heuristic cho thuật toán A*, giúp định hướng tìm kiếm hiệu quả và giảm đáng kể số trạng thái mở rộng.

3. **Thực nghiệm và đo lường khách quan**: Xây dựng thư viện `benchmark` đo đạc độc lập, chuẩn hóa quy trình đo thời gian (*median-of-3*) và bộ nhớ RAM đỉnh (`tracemalloc`) trên bộ dữ liệu kiểm thử 10 màn chơi phân tầng độ khó, trích xuất số liệu tự động ra bảng LaTeX và 4 biểu đồ vector.

4. **Giao diện trực quan**: Xây dựng giao diện đồ họa 3D bằng thư viện Ursina (Panda3D), hỗ trợ phát lại hoạt ảnh lăn khối mượt mà và trực quan hóa toàn bộ tiến trình giải của các thuật toán.

7.2. Hướng phát triển trong tương lai

- **Cải tiến hàm Heuristic**: Nghiên cứu các kỹ thuật Heuristic nâng cao như Pattern Database (PDB) hoặc phân tích đồ thị tĩnh có xét đến chướng ngại vật và trạng thái cầu để thu hẹp hơn nữa không gian tìm kiếm trên các màn chơi quy mô lớn.

- **Tối ưu hóa giải thuật**: Ứng dụng các biến thể tìm kiếm nâng cao như IDA* (Iterative Deepening A*) hoặc Bidirectional Search (tìm kiếm hai chiều) nhằm tối ưu hơn nữa lượng bộ nhớ tiêu thụ khi mở rộng sang các màn chơi phức tạp có độ sâu nghiệm lớn.

8. Tài liệu tham khảo & Phụ lục (References & Appendix)

8.1. Tài liệu tham khảo

1. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach (4th Edition)*, Pearson, 2020.

2. Coolmath Games, *Bloxorz Original Flash Game*, https://www.coolmathgames.com/0-bloxorz.

3. Slide bài giảng môn *Cơ sở Trí tuệ nhân tạo*, Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên - ĐHQG-HCM.

8.2. Phụ lục: Công bố sử dụng Trí tuệ Nhân tạo (GenAI Disclosure)

Theo quy định về việc sử dụng AI có trách nhiệm và minh bạch của môn học:

- **Mục đích sử dụng**: Nhóm sử dụng công cụ Generative AI (Antigravity Assistant) để hỗ trợ:
  - Lên khung cấu trúc tài liệu LaTeX và kiểm tra cú pháp định dạng bảng biểu.
  - Xây dựng khung thư viện đo đạc hiệu năng (`benchmark`).
  - Tối ưu hóa code vẽ đồ thị Matplotlib.
- **Kiểm định**: Toàn bộ logic trò chơi, phần thuật toán BFS/DFS và số liệu thực nghiệm đều do các thành viên trong nhóm tự cài đặt, kiểm thử và đo lường trực tiếp trên máy tính.

8.2.1. Hỗ trợ của trí tuệ nhân tạo trong phần UCS/A*

Đã sử dụng Codex để hỗ trợ đọc yêu cầu và hợp đồng giao diện, tách các PR và điều chỉnh theo base của người phụ trách nhóm, soạn và rà soát mã chính sách chi phí, frontier ưu tiên và bộ giải UCS/A*, xây dựng kiểm thử, kiểm tra lập luận hàm ước lượng và biên soạn phần báo cáo tương ứng. Các yêu cầu chính gồm: xác định chi phí không đồng nhất có cận dưới dương; xử lý đường rẻ hơn và bản ghi cũ; chứng minh hàm ước lượng; đối chiếu với chi phí tối ưu còn lại trên màn nhỏ; và chuẩn bị nội dung tiếng Việt theo cấu trúc LaTeX của nhóm.

Các gợi ý được đối chiếu với mã nguồn thực tế, lời giải phát lại bằng mô hình trò chơi và phép tính độc lập trên đồ thị. Số đo thời gian hoặc bộ nhớ không được tự đặt. Phần đóng góp này giữ phạm vi ở các mục được phân công và bảo toàn nội dung của thành viên khác. Người nộp chịu trách nhiệm kiểm tra, hiểu và giải thích được thuật toán, trọng số, chứng minh và giới hạn của phần mình phụ trách.